M02 - SQL Foundations

14-lesson beginner-sufficient learner book - RC1

How to use each lesson

Read the explanation, reproduce the small example, complete the matching guided lab, then close the book for the independent task. A lesson is not mastered merely because the example works.

DE02-01 - Database, table, row, column and query mental model

Why this matters	SQL sends declarative questions to a database; the result grid is only the output.

Core explanation

• A database stores related objects such as tables and views.

• A query describes the result you want.

• SELECT chooses expressions/columns; FROM chooses source objects.

• The output grain depends on your SELECT, joins and grouping.

Concrete example

```
SELECT order_id, region FROM sales_orders LIMIT 5;
```

Common failure	Do not confuse a query result with the stored table itself.
Proof before move-on	Run a first query and explain input grain vs result grain.

Explain aloud

In 60-90 seconds explain database, table, row, column and query mental model, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-02 - SELECT, aliases and LIMIT

Why this matters	Start with small, inspectable queries before adding logic.

Core explanation

- Select only columns needed for the question.
- Aliases make calculated outputs readable.
- LIMIT is useful for inspection, not for business totals.
- Use deterministic ORDER BY when "top" or "first" matters.

Concrete example

```
SELECT order_id, qty*unit_price AS gross_sales FROM sales_orders LIMIT 10;
```

Common failure	Do not report a limited sample as if it were the whole table.
Proof before move-on	Return three named columns plus one calculated alias.

Explain aloud

In 60-90 seconds explain select, aliases and limit, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-03 - WHERE and comparison operators

Why this matters	Filtering changes which rows survive before grouping.

Core explanation

• Use =, <>, >, >=, <, <= carefully.

• Text comparisons are quoted.

• Filter on business meaning, not accidental row positions.

• Validate filtered row counts.

Concrete example

```
SELECT * FROM sales_orders WHERE status='Completed';
```

Common failure	Do not use HAVING for ordinary row filtering.
Proof before move-on	Filter completed orders above a value threshold and count them.

Explain aloud

In 60-90 seconds explain where and comparison operators, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-04 - AND, OR, NOT and parentheses

Why this matters	Boolean logic must match the business rule exactly.

Core explanation

• AND narrows; OR broadens.

• Parentheses make precedence explicit.

• NOT can invert conditions but should remain readable.

• Translate the rule into English before SQL.

Concrete example

```
WHERE status='Completed' AND (region='East' OR region='North')
```

Common failure	A missing pair of parentheses can change the result without causing a syntax error.
Proof before move-on	Translate a two-part business rule into SQL and explain it aloud.

Explain aloud

In 60-90 seconds explain and, or, not and parentheses, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-05 - ORDER BY, DISTINCT and result inspection

Why this matters	Sorting and deduplication answer different questions.

Core explanation

• ORDER BY changes presentation order, not stored data.

• DISTINCT removes duplicate result combinations.

• DISTINCT can hide a join problem if used as a bandage.

• Use explicit sort direction and tie-breakers when needed.

Concrete example

```
SELECT DISTINCT region FROM sales_orders ORDER BY region;
```

Common failure	Do not add DISTINCT merely to make duplicates disappear.
Proof before move-on	Return unique statuses and explain why DISTINCT is valid there.

Explain aloud

In 60-90 seconds explain order by, distinct and result inspection, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-06 - NULL, IS NULL and COALESCE

Why this matters	Missing values need explicit handling; = NULL is not normal SQL logic.

Core explanation

• Use IS NULL / IS NOT NULL.

• COALESCE replaces NULL for a specific output purpose.

• Replacement values can change meaning.

• COUNT(column) ignores NULL, COUNT(*) does not.

Concrete example

```
SELECT COUNT(*) total_rows, COUNT(sales_rep) rows_with_rep FROM sales_orders;
```

Common failure	Do not replace unknown data with zero unless zero really means none.
Proof before move-on	Find missing sales_rep values and compare COUNT(*) vs COUNT(sales_rep).

Explain aloud

In 60-90 seconds explain null, is null and coalesce, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-07 - Arithmetic expressions and safe calculations

Why this matters	Calculated fields should make units and denominator rules explicit.

Core explanation

- Parentheses make calculation order clear.
- Use decimal-safe logic for rates.
- Guard denominators against zero.
- Name outputs with meaningful aliases.

Concrete example

qty*unit_price*(1-discount_pct) AS net_sales

Common failure	Do not mix percent 15 with decimal 0.15 without defining the stored convention.
Proof before move-on	Calculate net sales and manually validate two rows.

Explain aloud

In 60-90 seconds explain arithmetic expressions and safe calculations, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-08 - Aggregate functions

Why this matters	Aggregates collapse many rows into summaries.

Core explanation

• COUNT, SUM, AVG, MIN, MAX are common.

• Know whether you are counting rows, IDs or distinct entities.

• AVG ignores NULL in the expression.

• State result grain after aggregation.

Concrete example

```
SELECT COUNT(*) orders, SUM(net_sales) total_sales FROM ...;
```

Common failure	Do not call COUNT(*) customer_count on an order table.
Proof before move-on	Compute total and average completed sales with correct meaning.

Explain aloud

In 60-90 seconds explain aggregate functions, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-09 - GROUP BY and grouped grain

Why this matters	GROUP BY changes result grain to the grouping keys.

Core explanation

- Every group is one output bucket.
- Grouped result grain should be stated explicitly.
- Aggregate only measures meaningful at that grain.
- Reconcile grouped totals back to a grand total.

Concrete example

```
SELECT region, SUM(net_sales) FROM sales_orders GROUP BY region;
```

Common failure	Do not select ungrouped descriptive columns without a defined rule.
Proof before move-on	Return sales by region and reconcile to overall sales.

Explain aloud

In 60-90 seconds explain group by and grouped grain, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-10 - HAVING after aggregation

Why this matters	HAVING filters groups after aggregation.

Core explanation

• WHERE filters rows before grouping.

• HAVING filters grouped results.

• Using both can be correct when they do different jobs.

• State which rows are excluded before the aggregate.

Concrete example

```
WHERE status='Completed' GROUP BY region HAVING SUM(net_sales)>5000
```

Common failure	Do not move a row-level status filter into HAVING.
Proof before move-on	Return only regions above a threshold after filtering completed orders.

Explain aloud

In 60-90 seconds explain having after aggregation, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-11 - CASE for business categories

Why this matters	CASE creates rule-based derived categories inside a query.

Core explanation

- Order conditions from specific to general.
- Include an ELSE so unmatched rows are visible.
- Keep business thresholds documented.
- Validate category counts.

Concrete example

```
CASE WHEN net_sales>=2500 THEN 'High' WHEN net_sales>=1000 THEN 'Medium' ELSE 'Low' END
```

Common failure	Do not bury changing business thresholds as unexplained magic numbers.
Proof before move-on	Classify orders and reconcile category counts to total rows.

Explain aloud

In 60-90 seconds explain case for business categories, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-12 - String and date basics

Why this matters	Business data often needs small transformations before grouping/filtering.

Core explanation

• TRIM and case functions can standardize labels.

• Date extraction should use date-aware functions where possible.

• String cleanup is not a substitute for source remediation.

• Dialect functions vary across engines.

Concrete example

Use TRIM(region) and extract month from an order date using your engine syntax.

Common failure	Do not slice date strings manually if the column is a real date type.
Proof before move-on	Standardize one text field and group by month.

Explain aloud

In 60-90 seconds explain string and date basics, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-13 - Simple subqueries and IN/EXISTS awareness

Why this matters	One query result can become an input to another, but clarity matters.

Core explanation

• A scalar subquery returns one value.

• IN/EXISTS can express membership conditions.

• Validate the inner query independently first.

• Deep nesting belongs in later advanced SQL.

Concrete example

Return orders above the overall average order value using a scalar subquery.

Common failure	Do not debug two nested layers at the same time.
Proof before move-on	Run the inner query first, then use it in one outer filter.

Explain aloud

In 60-90 seconds explain simple subqueries and in/exists awareness, the input/output grain or program state, and one way a plausible-looking result could still be wrong.

DE02-14 - Debugging and validation workflow

Why this matters	SQL mastery means proving the query answers the question, not merely that it runs.

Core explanation

• Read errors from the first failing clause.

• Check row counts before and after filters.

• Validate one metric with an independent query.

• Save a minimal reproducible query when stuck.

Concrete example

Compare SUM(net_sales) by region with a separate grand-total query; grouped sums should reconcile.

Common failure	A syntactically valid query can still be logically wrong.
Proof before move-on	Solve an integrated foundation query and write three validation checks.

Explain aloud

In 60-90 seconds explain debugging and validation workflow, the input/output grain or program state, and one way a plausible-looking result could still be wrong.